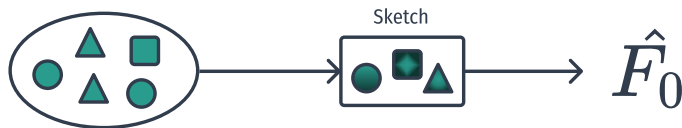
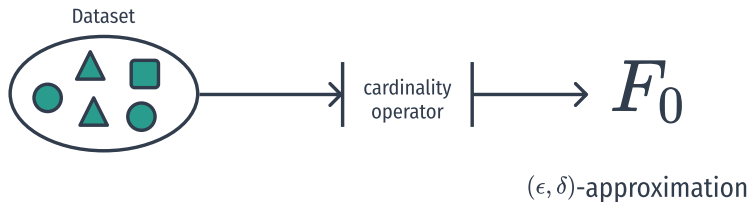


Stima della cardinalità nei flussi di dati distribuiti:

merge e compatibilità semantica
degli algoritmi di sketching

Daniele Ferroli

25 aprile 2026



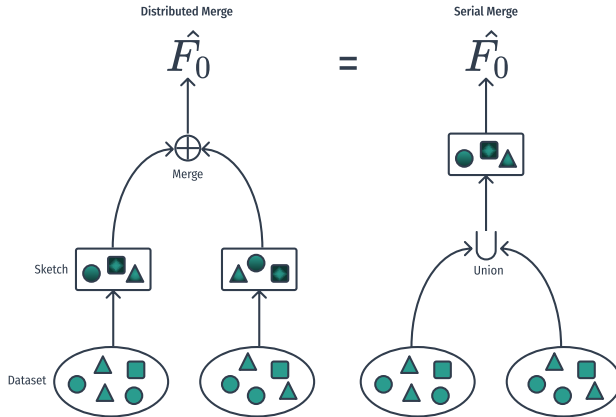
where (ϵ, δ) -approximation means

$$0 < \epsilon, \delta < 1 : \mathbb{P}[|F_0 - \hat{F}_0| > \epsilon F_0] < \delta$$

- sicurezza di rete / monitoraggio / telemetria
- conteggio di like, visitatori unici, page ranking
- DBMS
- ...

- implementazione degli algoritmi PC, LL, HLL, HLL++
- framework sperimentale modulare, estendibile
- analisi empirica sistematica
 - ▶ focus in particolare sul merge

Merge



- **assunzioni:** stessi parametri algoritmici, hash, seed
- **motivazioni:** sistemi distribuiti eterogenei

- implementazione degli algoritmi PC, LL, HLL, HLL++
- framework sperimentale modulare, estendibile
- analisi empirica sistematica
 - ▶ focus in particolare sul merge
 - classificazione operativa
 - **degradazione** dovuta a normalizzazione
 - ...

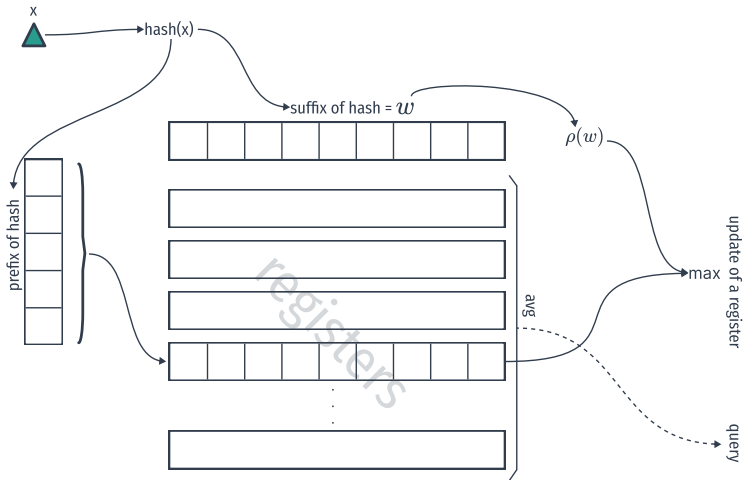
Sketch come struttura dati dinamica



- **update:** operazione $O(1)$
- **query:** come funziona
- **vincoli:**
 - ▶ unbiased ($F_0 \approx \hat{F}_0$)
 - ▶ update time $O(1)$
 - ▶ small memory ...

<u>lower bound</u>	<u>upper bound</u>
$\Omega(\epsilon^{-2} + \log N)$	$O(m \cdot \log N)$
$\Theta(\epsilon^{-2} + \log N)$	

Sketch LogLog



update
query:

$$\hat{F}_0 = \alpha_m \cdot m \cdot 2^A$$

- HyperLogLog
- HyperLogLog++
- UltraLogLog
- ...

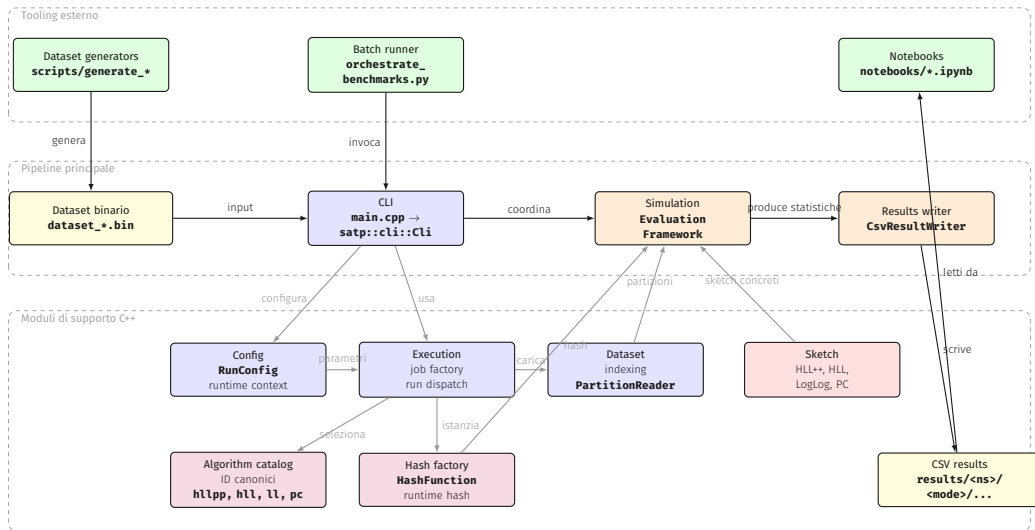
media armonica, correzione del bias
sparse vs dense, **correzioni empiriche**
stima più efficiente, registri più informativi

Compatibilità semantica del merge

Il merge è corretto solo se le due sketch descrivono i dati con la **stessa semantica**: algoritmo, parametri e hash devono essere compatibili.

valid	recoverable	invalid
Stessa semantica su entrambe le sketch. Azione: merge diretto.	Non direttamente compatibile, ma normalizzabile. Azione: riduzione e merge.	Non esiste un merge semanticamente corretto. Azione: rifiuto.

Framework



Dataset

- un file di un dataset è diviso in p partizioni
- ogni partizione contiene n elementi totali e d elementi distinti
- la partizione è strutturata a prefisso con dei blocchi $\rho = \frac{n}{d}$

	blocco 1			blocco 2			blocco 3			blocco 4		
t	1	2	3	4	5	6	7	8	9	10	11	12
x_t	a	a	a	b	a	b	c	a	c	d	b	c
b_t	1	0	0	1	0	0	1	0	0	1	0	0
$F_0(t)$	1	1	1	2	2	2	3	3	3	4	4	4

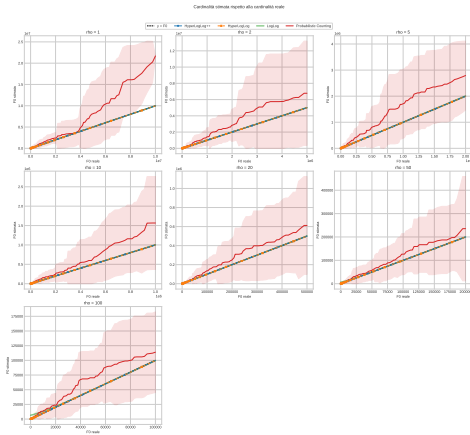
Tabella 1: Partizione con $n = 12$, $d = 4$ e $\rho = n/d = 3$

esempio

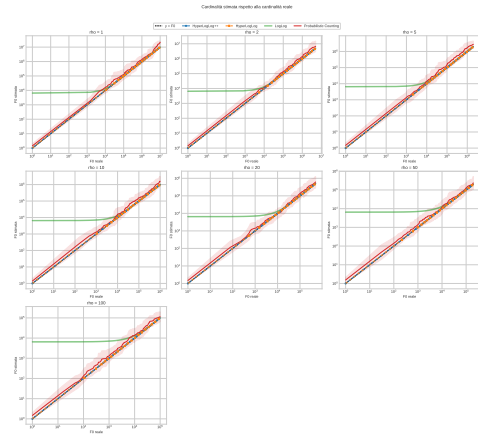
un dataset **binario** e **compresso** da 50 partizioni, con $n = 10^7$ e $\forall d$ occupa $\approx 1.7\text{GB}$!!
aumentare di un ordine di grandezza $n \Rightarrow$ **10x** sulla dimensione

- Cardinalità stimata rispetto alla cardinalità reale
- Confronto tra diverse precisioni per lo stesso algoritmo
- Caso *recoverable* durante un operazione di merge

Cardinalità stimata rispetto alla cardinalità reale



scala lineare



scala logaritmica

$$n = 10^7, \rho \in \{1, 2, 5, 10, 20, 50, 100\}, \text{seed} = 21041998.$$

Caso recoverable durante un operazione di merge

Framework modulare ed estendibile

Accuratezza HLL++ risulta l'algoritmo migliore

Merge distribuito caso omogeneo = seriale

Compatibilità necessario l'uso degli stessi parametri per il merge

Recoverable necessario normalizzare $\implies$ degrado stima

- dataset sintetico vs empirico
- merge non pairwise con topologie diverse
- varianti di sketch su altri problemi:
 - ▶ membership
 - ▶ moments-frequency
- ...

Grazie per l'attenzione

Domande ?